

ID: 17CE049

Subject : CE473 Machine Learning

Practical 4: Binary Logistic Regression (Employee Retention Ration) Instructions

1. Now do some exploratory data analysis(EDA) to figure out which variables have direct and clear impact on employee retention (i.e. whether they leave the company or continue to work)
2. Plot bar charts showing impact of employee salaries on retention
3. Plot bar charts showing corelation between department and employee retention
4. Now build logistic regression model using variables that were narrowed down in step 1
5. Measure the accuracy of the model

In [16]:

```
import pandas as pd
from matplotlib import pyplot as plt
%matplotlib inline
import seaborn as sns
```

In [17]:

```
df = pd.read_csv("HR_comma_sep.csv")
df.head()
```

Out[17]:

	satisfaction_level	last_evaluation	number_project	average_montly_hours	time_spend_compa
0	0.38	0.53	2	157	
1	0.80	0.86	5	262	
2	0.11	0.88	7	272	
3	0.72	0.87	5	223	
4	0.37	0.52	2	159	

Data Analysis

In [18]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14999 entries, 0 to 14998
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   satisfaction_level      14999 non-null  float64
1   last_evaluation         14999 non-null  float64
2   number_project          14999 non-null  int64
3   average_monthly_hours  14999 non-null  int64
4   time_spend_company     14999 non-null  int64
5   Work_accident           14999 non-null  int64
6   left                   14999 non-null  int64
7   promotion_last_5years  14999 non-null  int64
8   Department              14999 non-null  object
9   salary                  14999 non-null  object
dtypes: float64(2), int64(6), object(2)
memory usage: 1.1+ MB
```

In [19]:

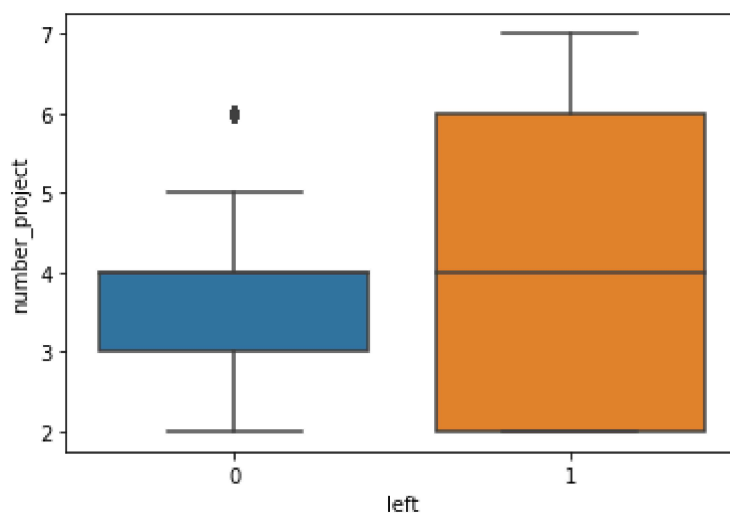
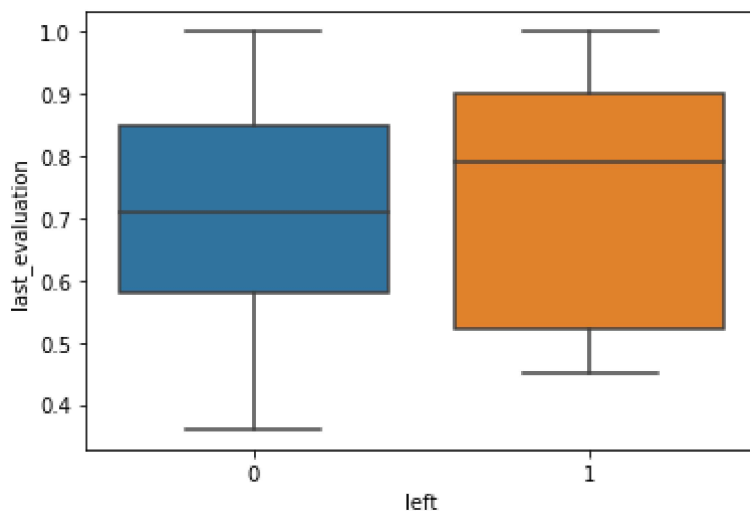
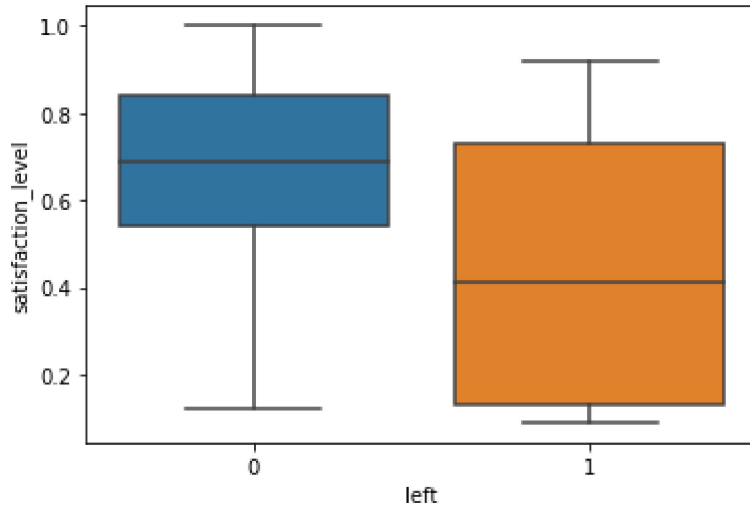
```
df.isnull().sum()
```

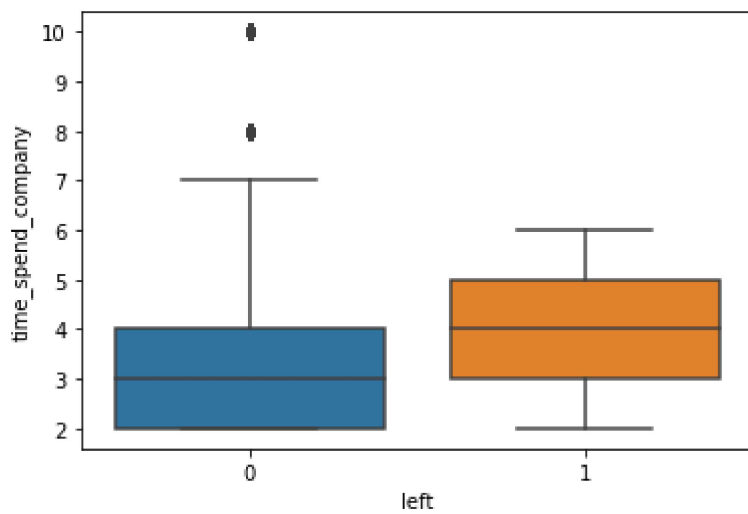
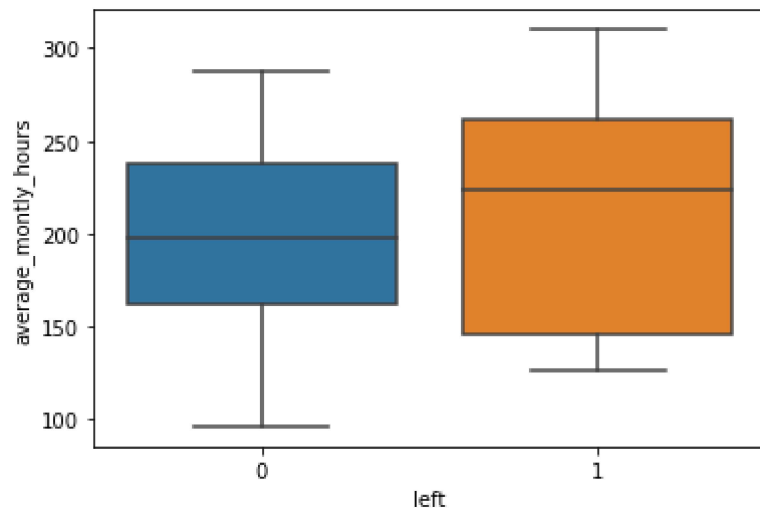
Out[19]:

```
satisfaction_level      0
last_evaluation          0
number_project           0
average_monthly_hours   0
time_spend_company      0
Work_accident            0
left                     0
promotion_last_5years    0
Department               0
salary                   0
dtype: int64
```

In [20]:

```
col = df.columns  
col=col.drop(["left", "Department", "salary", "Work_accident", "promotion_last_5years"])  
for c in col:  
    sns.boxplot(x="left", y=c, data=df)  
    plt.show()
```





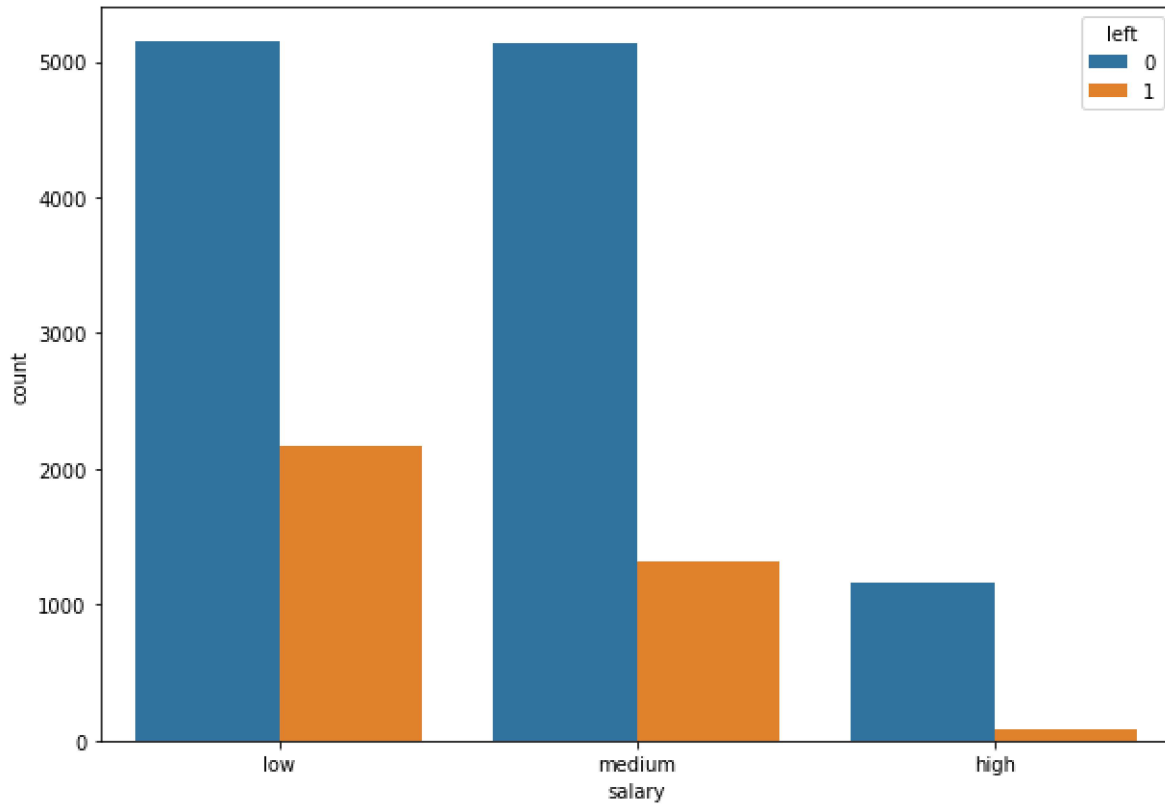
Bar chart to show impact of employee salaries on retention

In [21]:

```
plt.figure(figsize=(10,7))  
sns.countplot(x="salary",hue="left",data=df)
```

Out[21]:

<matplotlib.axes._subplots.AxesSubplot at 0x1899df72dc8>



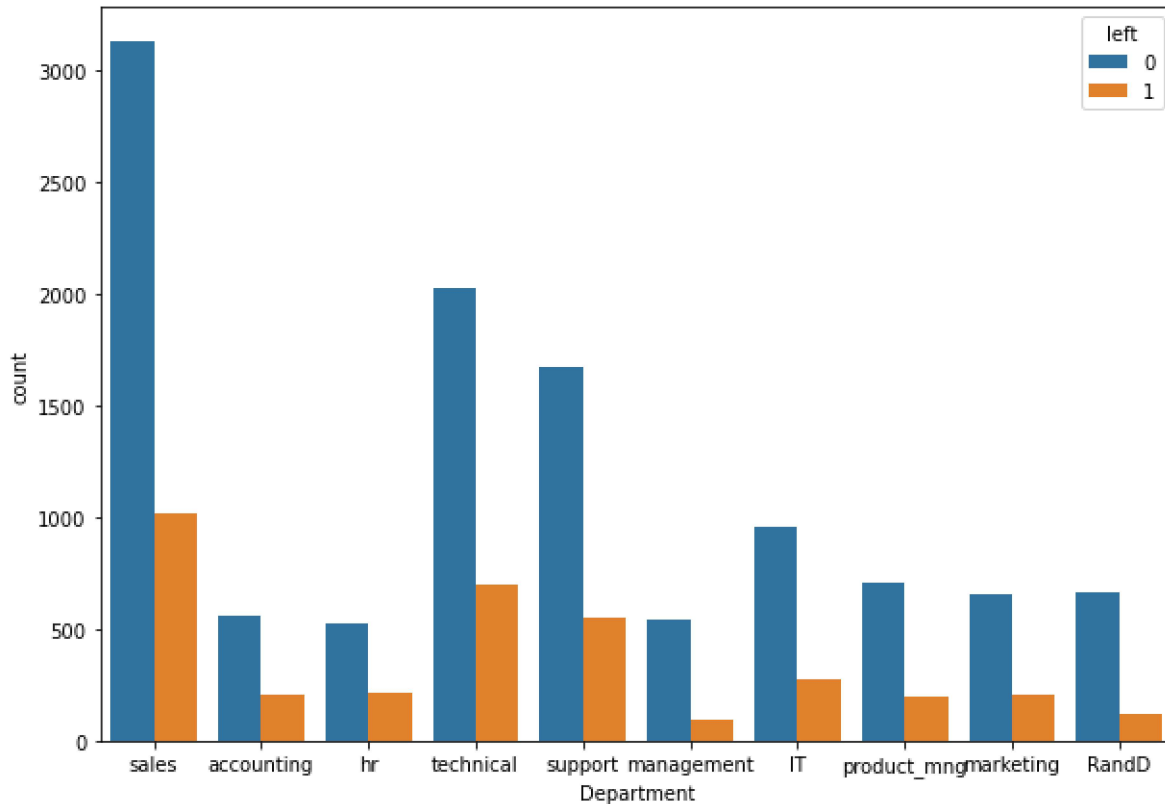
Bar chart to show correlation between department and employee retention

In [22]:

```
plt.figure(figsize=(10,7))
sns.countplot(x="Department",hue="left",data=df)
```

Out[22]:

<matplotlib.axes._subplots.AxesSubplot at 0x1899e07c7c8>



Creation of Logistic regression model

In [23]:

```
sal = pd.get_dummies(df['salary'],prefix='sal')
df = pd.concat([df,sal],axis=1)
X = df[['satisfaction_level','average_monthly_hours', 'promotion_last_5years','sal_high', 'sal_medium']]
y = df['left']
```

In [24]:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
```

In [25]:

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
```

In [26]:

```
model.fit(X_train,y_train)
```

Out[26]:

```
LogisticRegression()
```

In [27]:

```
X_test
```

Out[27]:

	satisfaction_level	average_monthly_hours	promotion_last_5years	sal_high	sal_low	sal_r
14796	0.40	139	0	0	1	
13110	0.89	233	0	0	0	
13358	0.57	250	0	0	1	
3038	0.12	260	0	0	0	
6930	0.58	223	0	0	1	
...	...	...	...	...	...	
2200	0.89	272	0	0	1	
1004	0.09	247	0	0	0	
8213	0.91	260	0	0	1	
9752	0.69	156	0	0	1	
3892	0.85	153	0	0	1	

4500 rows × 6 columns

In [28]:

```
y_test
```

Out[28]:

```
14796    1
13110    0
13358    0
3038     0
6930     0
..
2200     0
1004     1
8213     0
9752     0
3892     0
Name: left, Length: 4500, dtype: int64
```

In [29]:

```
model.predict(X_test)
```

Out[29]:

```
array([0, 0, 0, ..., 0, 0, 0], dtype=int64)
```

Accuracy of the model

In [30]:

```
print("Model accuracy is ",model.score(X_train,y_train))
```

```
Model accuracy is  0.7773121249642823
```